

Your music. Your cards. Your show.

Player guide, host handbook and a plain-English look inside the game engine



Hazz Music Bingo v1.0

Windows x64 • Illustrated reference edition • 3 October 2026

PREPARE
Choose the music

PLAY
Host the room

VERIFY
Celebrate winners

From opening the ZIP to running a complete themed night, this manual explains what to click, what your players do, what gets saved and how the 60-song / 60-card engine works.

What is included

Real application screenshots, clear winning-pattern diagrams, a worked card-generation example, probability tables, troubleshooting and an event-day checklist.

Based on release v1.0, source tag v1.0. Example labels and songs in screenshots are demonstration data, not supplied music.

Find your way around

You do not need to understand the mathematics to host a game.

Start here	Pages
What players do and the three winning patterns	3-4
The complete event workflow and installation	5-6
Host console, library, MP3 tags and folder selection	7-9
Create a game, design and print cards	10-12
Sold cards, playback and winner checking	13-15
Audience themes and eight game buttons	16-17
A complete worked 1970s night	18
Save, resume, reset, back up and move computers	19-20
Inside the engine: randomisation and card construction	21-24
Balance, winner logic and probability	25-27
Timing, troubleshooting and event checklist	28-30
Glossary, sources, compatibility and licence	31

A useful first route

Read pages 3-6, then follow the worked 1970s example on page 18. Keep page 30 nearby at your event. Read pages 21-27 when you want to understand why the cards behave as they do.

Three things to remember: generate the cards before saving your prepared game; apply the sold-card range before relying on automatic winner detection; never reset played songs between Line, Four Corners and Full House.

How music bingo is played

Think of ordinary bingo, with song titles instead of numbered balls.

- 1. Take your card and a pen.** Your card has a 5 by 5 grid containing 25 song titles. The host uses a pool of 60 songs. Not every song played will be on your card.
- 2. Listen to each clip.** When you recognise a song printed on your card, mark that square. The host can repeat the previous clip or show the played-song list.
- 3. Follow the current prize rule.** Watch the audience message or listen to the host. A Line, Four Corners and Full House require different arrangements.
- 4. Call bingo when the rule is complete.** Give the host your card number and allow them to check the game code and marked squares.
- 5. Keep the same marks for the next prize.** When the host moves to another rule within the same game, keep all previous marks. Do not wipe the card clean.

No free middle square

All 25 squares contain songs. The centre only counts after its song has been played. A diagonal does not count as a Line in v1.0.

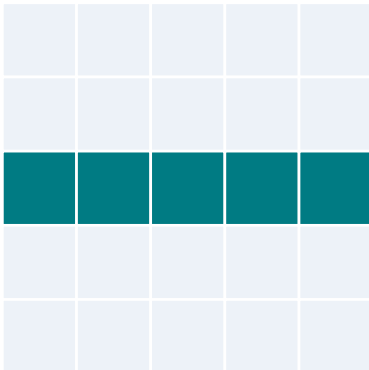
Agree the house rules before starting

The host decides prizes, how players call, whether hints or the audience song display are used, and how simultaneous claims are handled. The software checks recorded song progress; it does not decide your prize policy or identify the person holding a card.

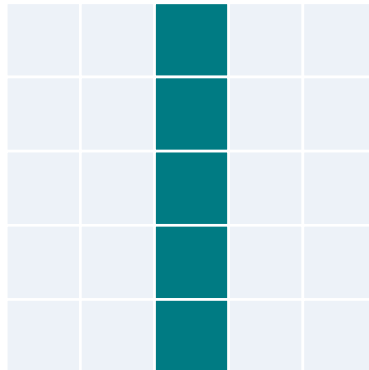
More than one card can win on the same song. That is a normal possibility, not evidence that a card was printed incorrectly.

The three winning patterns

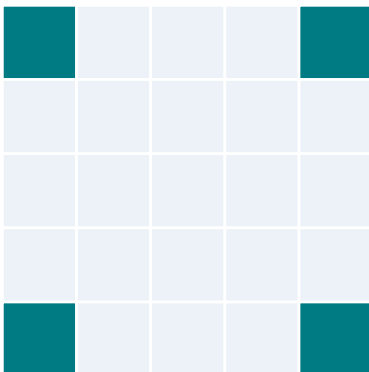
Filled squares below mean that those songs have been played.



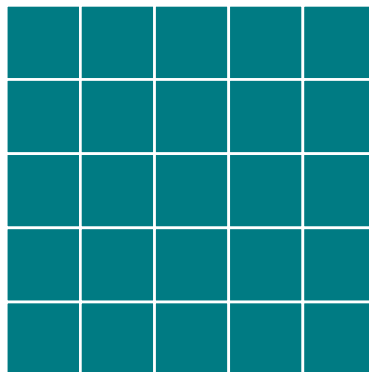
LINE: horizontal



LINE: vertical



FOUR CORNERS



FULL HOUSE

Line: any complete row or any complete column. There are five rows and five columns, so ten possible lines on a card. You need one of them, not a row and a column together.

Four Corners: the top-left, top-right, bottom-left and bottom-right songs. A completed line elsewhere does not satisfy this rule.

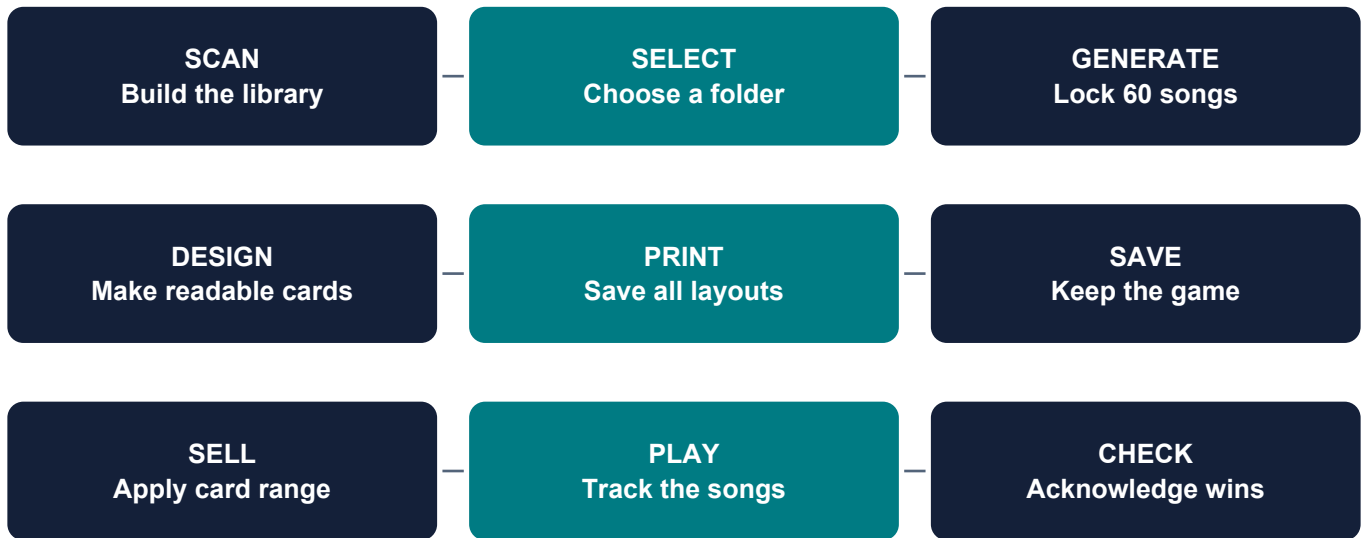
Full House: every one of the 25 songs on that card. There is no substitute for a missing square.

The host chooses the next prize

Line → Four Corners → Full House is a common sequence. The app does not advance automatically. Four Corners might already be complete when the host selects it; the app will report that immediately.

The whole event in one page

Prepare once; run the prizes using the same locked game.



1. Before printing Check the music is available and titles are correct. Generate the intended song pool, then open Preview / Print Cards to create its exact 60 layouts.

2. Before guests play Sell a known contiguous range of card numbers, apply that range, select Line, check sound and display, and save the prepared state.

3. During the show Use Play Next Song. Check claims against the printed game code. Acknowledge the winner(s), then choose the next rule without resetting progress.

4. After the show Save a portable copy if required and close the game. The local history retains its cards and progress. Back up the database and any separate themes/button assets.

One game is not the whole library

A game is a fixed set of 60 songs, its cards and its progress. Scanning more music or selecting another source folder does not change that existing game.

Get the app running

Windows x64; no separate .NET installation is needed.

- 1. Download the Windows ZIP.** Use the v1.0 release page. Choose Hazz-Music-Bingo-v1.0-Windows-x64.zip, not the source archive.
- 2. Extract the entire ZIP.** Open the extracted folder. Keep TagLibSharp.dll next to HazzMusicBingo.exe, and retain the documentation and licences.
- 3. Run HazzMusicBingo.exe.** The first launch starts with an empty library. Later launches reopen the active local game. The executable is unsigned.
- 4. Connect your event equipment.** Choose the Windows speaker output and connect an optional second monitor before opening the audience window. A printer is optional; Microsoft Print to PDF can create card PDFs.

Upgrading from an earlier version

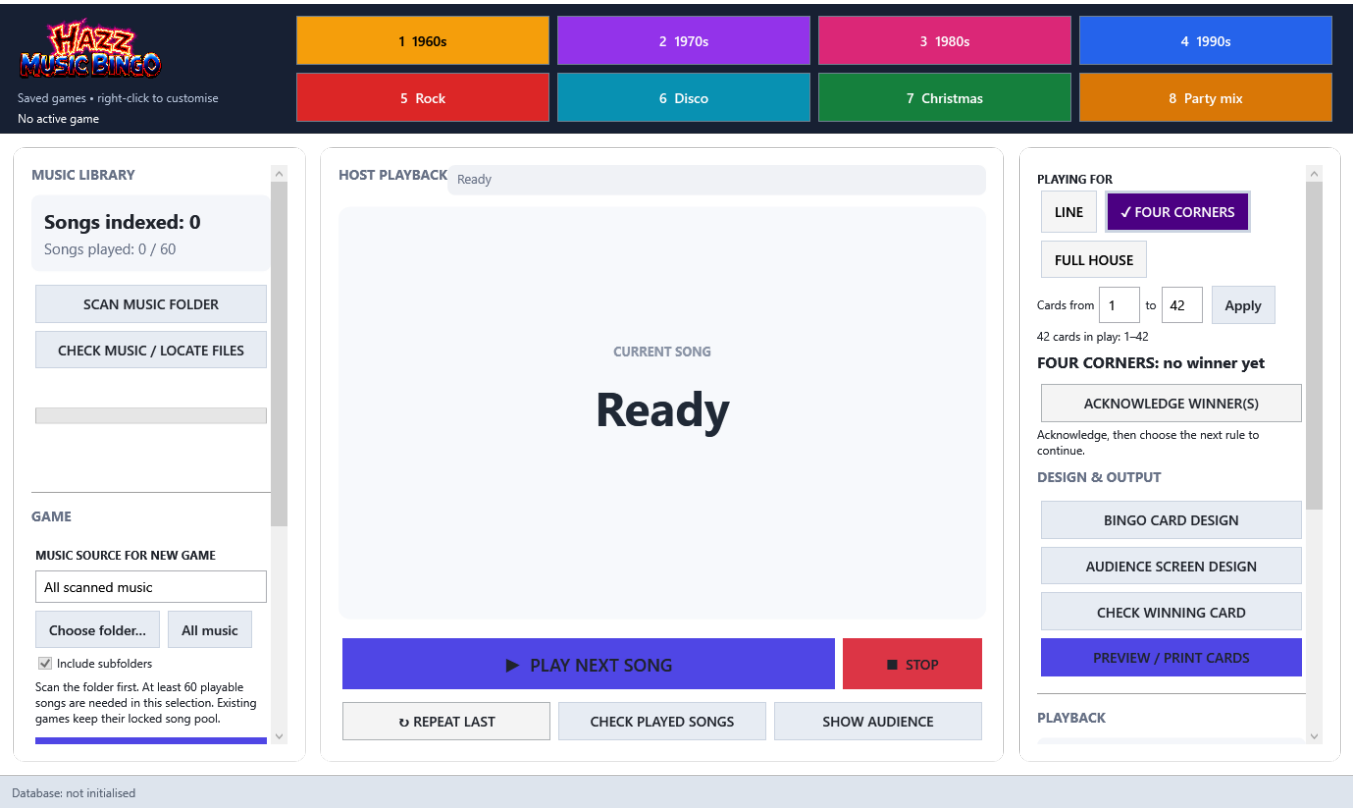
Close the old app. Back up your data, then extract the new version into a new folder. Using the same Windows account normally preserves the library and games because they live separately from the EXE. Keep only one instance open while hosting.

Requirement	Practical meaning
Music	Your own accessible music files; at least 60 playable scanned songs in the selected source.
Display	Host window minimum: 1120 × 680 logical pixels. Windows scaling affects usable space.
Data folder	%LOCALAPPDATA%\HazzMusicBingo, unless you deliberately use a separate profile.
Card output	A physical printer or an installed PDF printer. Test one sample before the full run.

Download: [GitHub release v1.0](#). The release also provides source and SHA-256 checksums.

The host console

Screenshot from the app; the eight labels shown are example assignments.



The real host layout. The side panels scroll when controls extend below the window.

Area	What you do here
Header	Click one of eight saved-game buttons; right-click to configure it. The logo and current game status identify the session.
Left panel	Scan/check music, choose the generation source, generate a game, save/load, reset or reopen history. Scroll down for lower controls.
Centre	Read the current title/artist, play the next clip, stop, repeat or display the alphabetical played list.
Right panel	Select the winning rule, apply the sold range, acknowledge winners, check cards, design the audience/cards and print.

If a control is out of sight

Scroll inside its left or right panel. The centre playback controls stay separate. A selected rule has a check mark and purple background; an active saved-game button has a gold border/play marker.

Scan music and read MP3 tags

Scanning adds to the library; it does not empty it first.

- 1. **Choose SCAN MUSIC FOLDER.** Pick the folder that contains your recordings. Scanning includes its subfolders. Wait for the completion message.
- 2. **Review the titles.** MP3 title and artist tags are preferred. Missing fields use the filename fallback independently. A file named Artist - Song Title.mp3 is a useful fallback.
- 3. **Run CHECK MUSIC / LOCATE FILES.** Check the entire library by clearing Current game only when necessary. Resolve unavailable files before generating.

File or condition	What happens
MP3 tags	ID3v1 and ID3v2 are supported, including Unicode and multiple performers. The app reads tags; it never writes them.
Missing title or artist	Only the missing/unreadable field falls back to the filename. A good tagged title can be used with a filename-derived artist.
Other formats	WAV, WMA, M4A, AAC, FLAC and OGG are indexed; labels still come from filenames. Actual playback depends on Windows decoding support.
No readable duration	The track remains indexed but is excluded from new-game selection until it is playable and rescanned/relinked.

Do metadata cleanup before printing

Rescanning updates shared library labels for existing song IDs. Existing game displays and reprints can therefore show new labels. Do not change names or substitute recordings after distributing cards.

Scanning is additive and does not remove obsolete entries. Separate copies of the same recording at different paths can be separate tracks: the app does not recognise audio duplicates by listening to them.

Make a 70s-only game

A source filter limits generation. It does not delete the rest of your library.



- 1. Scan the 70s folder.** A successful scan automatically selects that folder for the next new game and ticks Include subfolders.
- 2. Check MUSIC SOURCE FOR NEW GAME.** Under GAME, confirm the displayed path. Choose folder... can select another already-scanned folder; it does not scan that folder itself.
- 3. Choose the scope.** Tick Include subfolders to include child folders. Untick it for songs directly inside the selected folder.
- 4. Generate.** The app only selects available indexed tracks within that scope. It requires at least 60. A neighbouring folder such as 70s-other is not included merely because its name begins the same way.

Your selection	Result
70s folder, children off	Only files directly in 70s.
70s folder, children on	Files in 70s and its descendant folders.
All music	Available songs anywhere in the scanned library.
Only 47 available songs	Generation stops. It does not borrow 13 songs from elsewhere.

Check again after restarting

The source selector starts at All scanned music when the app restarts. Confirm it before generating. Existing games keep their locked pool regardless of this selector.

Generate, identify and lock a game

A game is the unit you save, print, play and recover.

Select GENERATE 60-SONG GAME once the source is correct. If another game is active, the app asks before closing it. It selects 60 available tracks and saves the pool with a playback order. If exactly 60 tracks are eligible, all 60 enter the pool; if more are eligible, a random selection is taken.

What gets locked, and when?

Stage	Saved result
Generate game	The 60 track identities, an initial shuffled playback order and a game/session code.
First Preview / Print Cards	All 60 exact card layouts. Printing fewer cards does not create a smaller master set.
Start each clip	The track becomes played once audio output has started successfully.
Apply range / choose rule	The in-play range and selected winning rule are stored with the game.
Save Game	A portable snapshot of the pool, progress, generated cards, game code and winning settings.

Generate cards before saving your master copy

A game saved before its cards exist has no layouts in that file. Loading it later can generate a new set. For an event you have printed, preserve a save made after card generation.

The printed game code distinguishes one session from another. Card 001 in a different game is a different layout. Imported copies can share a code, so history date and progress also matter. The code is an identifier, not an anti-fraud signature.

Design the player experience

The preview and print output share the same card renderer.

BINGO CARD DESIGN

Quick theme
Midnight — navy and gold

Apply theme

Card title
MUSIC BINGO

Header / subtitle

Font
Arial

Title size: 34 Header size: 18 Song size: 16

☒ Show B I N G O column letters
☒ Show artist as well as song title
☐ Bold song text
☐ Use text outline / stroke

Outline Colour Outline width (px): 2.0

COLOURS

Song Text Colour Title Colour

Page Background Square Background

GAME PREVIEW • CARD 001

MUSIC BINGO

B	I	N	G	O
Take On Me — a-ha	Africa — Toto	Billie Jean — Michael Jackson	Purple Rain — Prince	Jump — Van Halen
Beat It — Michael Jackson	Like a Virgin — Madonna	The Final Countdown — Europe	Tainted Love — Soft Cell	Under Pressure — Queen & David Bowie
Careless Whisper — George Michael	Thriller — Michael Jackson	Sweet Child o' Mine — Guns N' Roses	Eye of the Tiger — Survivor	Summer of '69 — Bryan Adams
Every Breath You Take — The Police	When Doves Cry — Prince	Don't Stop Believin' — Journey	Girls Just Want to Have Fun — Cyndi Lauper	Flashdance — Irene Cara
Sweet Dreams — Eurythmics	With or Without You — U2	Livin' on a Prayer — Bon Jovi	Walk This Way — Run-D.M.C.	I Wanna Dance with Somebody — Whitney Houston

Listen • Mark your songs • Call BINGO!

Card designer with an example layout. Font and artwork choices belong to the card design, not the audience theme.

Open BINGO CARD DESIGN. Choose an installed font and adjust the card title, subtitle, song size, artist visibility and colours. Use Classic, Midnight, Festival or Easy read as a starting theme, then personalise it.

You can change background artwork and opacity, grid thickness, square padding, alternate-row shading, alignment, text outlines and the footer. Long song labels wrap and shrink to fit. Large heading/footer text is limited, so keep it concise.

- 1. Preview real readability.** Use good contrast. Hiding artists and printing fewer cards per sheet can help more than simply increasing the font size.
- 2. Select Use Design.** This keeps the changes. Cancel discards the working edit. Save As creates a .hmbdesign preset; Load Design brings one back.
- 3. Keep the assets.** Ordinary design presets reference background files by path. Copy the artwork separately when moving a design to another computer, and install the same fonts.

Get the cards into players' hands

The print range and the sold-card range are separate choices.

PRINT CARDS

Game ABCDEF123456 • 60 saved layouts

First card number
1

Last card number
20

Cards per sheet
1 — largest text

Paper
A4

Orientation
Portrait

Page margin (5–30 mm)
10

☐ Ink saver (black on white)

Update preview

Print / Save PDF...

20 cards • 20 sheets per copy
Card numbers 001–020

To create a PDF, select Microsoft Print to PDF in the printer dialog. Copies and page range are selected there. Pages are fitted to the selected printer's printable area.

Four cards per sheet produces smaller text.
Print one sample before a full run.

Preview of a Music Bingo card (CARD 001):

GAME ABCDEF123456 • CARD 001

MUSIC BINGO

B	I	N	G	O
A very long song title that must remain complete on the printed card — Example Artist	Song 02 — Example Artist	Song 03 — Example Artist	Song 04 — Example Artist	Song 05 — Example Artist
Song 06 — Example Artist	Song 07 — Example Artist	Song 08 — Example Artist	Song 09 — Example Artist	Song 10 — Example Artist
Song 11 — Example Artist	Song 12 — Example Artist	Song 13 — Example Artist	Song 14 — Example Artist	Song 15 — Example Artist
Song 16 — Example Artist	Song 17 — Example Artist	Song 18 — Example Artist	Song 19 — Example Artist	Song 20 — Example Artist
Song 21 — Example Artist	Song 22 — Example Artist	Song 23 — Example Artist	Song 24 — Example Artist	Song 25 — Example Artist

Custom footer

Type text to find...

Print preview: select card numbers, review the sheets, then print or choose a PDF printer.

- 1. Open PREVIEW / PRINT CARDS.** The first request generates and stores the full 60-card set. Reopening or reprinting uses those saved layouts.
- 2. Choose first and last card.** The inclusive range 11–20 prints ten cards. Select 1, 2 or 4 cards per sheet; A4 or Letter; portrait or landscape; margins and optional ink saver.
- 3. Update preview and check a sample.** Match the physical printer settings to the preview. Printable margins may cause further scaling.
- 4. Print / Save PDF....** Choose your printer, or Microsoft Print to PDF and an output filename. The printer dialog's page range means sheets, not card numbers.

How much paper?

Sheets per copy = round up(cards printed ÷ cards per sheet). For 42 cards at four per sheet: round up(42 ÷ 4) = 11 sheets. The final sheet contains two cards. Sixty cards at four per sheet use 15 sheets.

Do not sell duplicate copies of the same numbered card as if they were different cards. They have identical songs and will qualify together; the app reports that card number, not how many paper copies exist.

Tell the app which cards are in play

Automatic detection needs the applied sold-card range.

- 1. Sell a known sequence.** For example, sell cards 1 through 42 from this game. Keep a record and avoid gaps where possible.
- 2. Enter first and last under PLAYING FOR.** Set 1 and 42, then press Apply. The app validates both endpoints and every intervening saved card number.
- 3. Read the confirmation.** The applied range and count appear underneath. Editing the boxes without Apply does not change the active range.
- 4. Select LINE.** The selected button is obvious. Open the audience screen and confirm its message before starting.

Example	Meaning
1 to 42	42 numbered cards in play: $42 - 1 + 1 = 42$.
11 to 20	10 cards in play, not 20.
1 to 60	All 60 distinct card numbers monitored.
1 to 20 and 30 to 40	Two separate ranges are not supported. A single 1-40 range would also count unsold cards 21-29.

Printing is not selling

Printing cards does not activate automatic tracking. You must apply the in-play range. Older game files without a range open with tracking off until you do this.

Manual Check Winning Card can inspect any saved card, including an unsold one. Automatic detection only considers the applied range. Applying a new range during play immediately reevaluates the current progress; use care when correcting sales information.

Play, repeat, stop and review

The computer records progress when a clip starts, not when it finishes.

Control	What it does
PLAY NEXT SONG	Chooses the next unplayed track from this game's saved order. A successfully started clip marks that track played.
REPEAT LAST	Replays the last track without adding a second played entry. Next can interrupt a repeat.
STOP	Stops sound immediately. It does not unmark a song whose clip already started.
Start music at	Choose 0, 30, 60, 90 seconds or another 30-second step. Next and Repeat use it. Short tracks fall back to 0 with a host notice. The setting is remembered after use, across games.
Clip length	Choose 10, 15, 20, 25, 30, 35, 40, 45 or 60 seconds. Clips start at the selected offset and fade over the final two seconds; shorter recordings finish sooner.
CHECK PLAYED SONGS	Shows an alphabetical paged list to the host and audience. Host page controls update the audience list. Closing it returns to the current-song view.

A comfortable host rhythm

Introduce the prize, play a clip, allow marking time, then continue. Repeat if needed. Use the played list to resolve uncertainty. The app does not choose prizes, pace the room for you or listen for a player shouting bingo.

Know what counts as played

A missing file or startup decoder/output failure leaves the song unplayed. Stopping after successful startup leaves it played. A later sound-device failure does not undo the flag. There is no individual song undo or skip control in v1.0.

The live winner panel updates from recorded progress. It is non-modal: a winning state does not automatically pause playback or advance the prize. You remain in control of sound and timing.

Handle a winner correctly

Check the paper card as well as the computer's winning state.

WINNER CHECK • GAME ABCDEF123456

Match the game code on the printed card before checking. Played songs are shown in green. No free centre square.

Card number

1

Line (horizontal or vertical)

Previous Card

Next Card

Check / Refresh

✓ Song 4 Artist	✓ Song 53 Artist	✓ Song 12 Artist	✓ Song 11 Artist	✓ Song 8 Artist
Song 1 Artist	Song 7 Artist	Song 47 Artist	Song 17 Artist	Song 5 Artist
Song 27 Artist	Song 19 Artist	Song 2 Artist	Song 9 Artist	Song 28 Artist
Song 35 Artist	Song 37 Artist	Song 21 Artist	Song 30 Artist	Song 38 Artist
Song 10 Artist	Song 33 Artist	Song 3 Artist	Song 29 Artist	Song 43 Artist

CARD 001: BINGO — pattern complete • 5/25 marked

Row 1

Saved-card checker: played squares are green, and the result names the completed pattern.

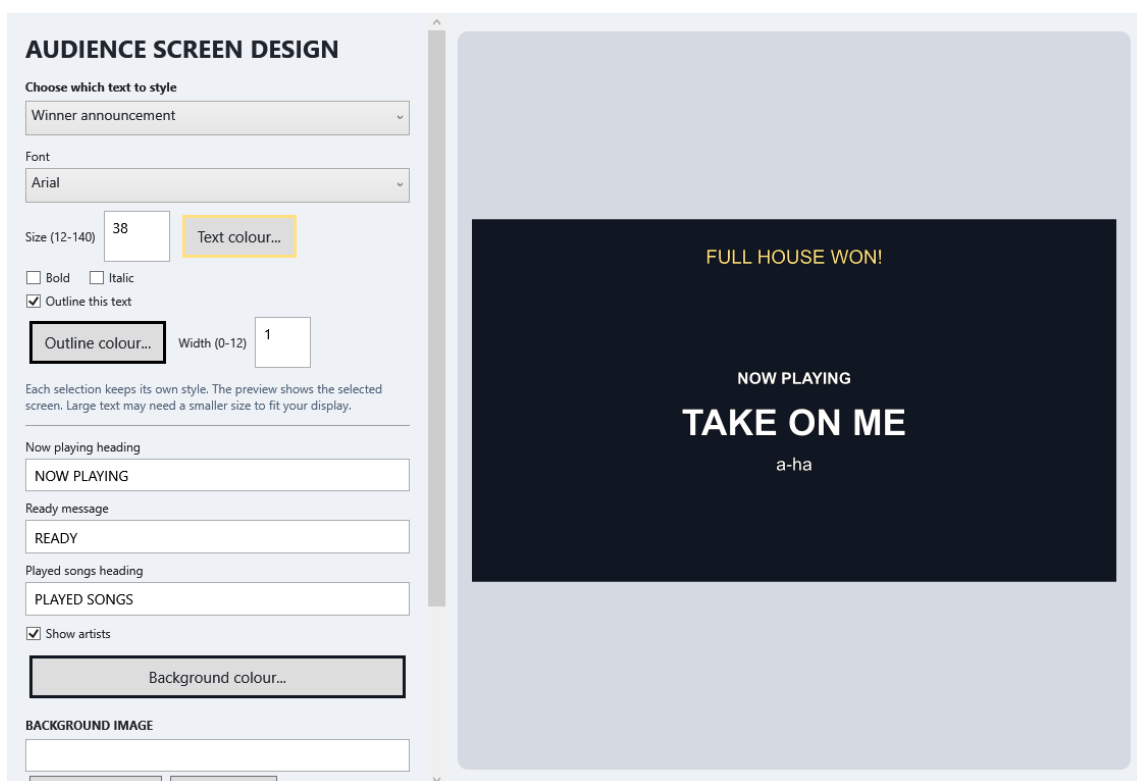
- 1. **Read the live winner panel.** It lists every qualifying in-play card for the selected rule. Multiple numbers may appear.
- 2. **Verify the claim.** Open CHECK WINNING CARD. Match the game code, enter the card number or use Previous Card / Next Card. Navigation follows saved card numbers and stops at the ends.
- 3. **Acknowledge the winner(s).** This clears the attention state for those card/rule pairs. The same pairs are not repeatedly flagged; newly qualifying cards still appear.
- 4. **Select the next rule.** Choose Four Corners and later Full House. All played songs remain. Do not press Reset between prizes.

The audience banner stays meaningful

A win changes the message to LINE WON!, FOUR CORNERS WON! or FULL HOUSE WON!
Acknowledgement leaves that message visible until the host chooses another rule. Song and played-list views still work.

Style every audience text element

Audience styling is separate from printed card styling.



The audience designer previews the selected text element. Playing-for and winner messages have separate styles.

Open AUDIENCE SCREEN DESIGN. Choose the text to style, then set its font, size (12-140), colour, bold, italic and outline colour/width (0-12). Use Design applies; Cancel discards. Save As / Load Design use .hmbaudience preset files.

Eleven choices cover the playing-for message, winner announcement, Now Playing heading, Ready, current title and artist, played heading, played-list titles and artists, page number and empty-list message. Changing one leaves the others alone. Old themes retain their defaults until edited.

Design for the back of the room

Use contrast and modest artwork opacity. Test long song names and the Full House message at the actual display size. A missing font can be substituted by Windows. Supply your own decade imagery; no music-night artwork or extra fonts is bundled.

Connect a second display before opening SHOW AUDIENCE. On a single display the audience window is a preview. Automatic monitor hot-plug recovery is not implemented. A background in an ordinary preset is a file reference; a game-button assignment makes its own artwork copy.

Eight saved-game buttons

One click changes the game and its audience theme.

GAME BUTTON

Button label

Saved game (.hmbgame)

Theme: Georgia • #111722 • no background image

 The button keeps its own copy of the save and background image. Clicking it resumes its game progress; switching never resets songs.

Assignment editor: your label, colour, saved game and audience style. Buttons start empty.

- 1. Prepare a saved game first.** Choose its music, generate and print its layouts, set the range/rule, and save a .hmbgame file.
- 2. Click an empty header button.** Browse to the file, give the button a label up to 32 characters and choose a colour.
- 3. Set Audience theme....** Pick fonts, colours and artwork, or use the current audience style. Save the button.
- 4. Click to switch.** Playback stops. First use imports the copied save; later uses resume that button's local game, including cards, played songs, rule, range and order.

Right-click to edit, store the current audience style or clear a button. Editing its label, colour or theme keeps resumed progress when the assigned file is unchanged. Assign a different save to start a different snapshot. Clearing a button does not erase its game history.

Copies are deliberate

The button owns copies of the save and background image, so moving the originals does not break it. Music is not copied. Changes to the original save do not update a button automatically; reassign it. Main audience-design edits need Store current audience style if you want that button to keep them.

Run a 1970s music night

A practical rehearsal from a folder of music to the final prize.

- 1. Prepare a source folder.** Put at least 60 suitable, distinct recordings in your 70s folder. Correct MP3 tags first. Avoid duplicate recordings under different filenames.
- 2. Scan and choose scope.** Scan that folder, confirm its path under Music Source for New Game and decide whether to include child folders. Check availability.
- 3. Generate and print.** Generate the pool. Design a clear card, then preview/print cards 1-42 at four per sheet: 11 sheets. Keep one sample and verify the game code.
- 4. Save the prepared game.** Apply sold range 1-42, select Line, then Save Game. Assign the save to a header button labelled 1970s.
- 5. Set the theme.** Use your own period artwork and an installed font in Audience theme. Keep the message readable. Test the speaker output and audience screen.
- 6. Run the prizes.** Play clips and allow marking time. Verify a Line claim, acknowledge and select Four Corners. Continue, then select Full House. Never reset between these prizes.
- 7. Finish deliberately.** Save the completed state if wanted, close the game and back up. Returning to its button later resumes this state: use Reset only if you intentionally want to replay those same cards.

If the corners are already complete

The moment you select Four Corners, the app checks all previously played songs. It can announce a winner immediately. This is correct: changing the rule does not start a fresh song history.

The app does not enforce a particular prize order or tie-break. Explain your own agreed rules before the first clip, especially whether a card may win more than one prize.

Save, load, resume or reset?

These actions are intentionally different.

Action	Cards / songs	Progress and order
Save .hmbgame	Exports the current pool, code and any generated layouts.	Saves progress, rule, range and acknowledgements. The order field is written for compatibility.
Load .hmbgame	Imports exact pool/layouts into a new local history record.	Restores played state; deliberately shuffles playback order again.
Reopen history	Reopens the same local database record.	Preserves progress and order exactly.
Assigned button: first click	Imports its owned snapshot.	Uses snapshot progress with a newly shuffled import order.
Assigned button: later click	Reopens its existing local game.	Preserves progress and order; applies its saved audience theme.
Reset Game to 0	Keeps the same pool and cards.	Clears played songs and acknowledgements; reshuffles order. Rule and sold range remain.
New / Close Game	Closes the active record, retaining history.	Progress/order remain available in history.

A game file is not an installation backup

It does not include music, card-design settings, audience-design settings, background images or the eight button assignments. Save those separately when moving an event or computer.

A save is a snapshot, not a continuously updated file. Save again when you want a new portable checkpoint. History is not time travel: resetting a record also changes what history will reopen. Use an earlier save or backup to restore an earlier state.

Backups, recovery and moved music

Use the built-in database backup for a consistent copy.

- 1. Back up the database.** Open GAME HISTORY / RECOVERY and choose Back up database.... Select a new filename; existing backup files are not overwritten.
- 2. Keep the other assets too.** Back up design settings, original artwork, game-shortcuts.json and game-buttons/ with the database. Keep your actual music separately backed up.
- 3. Recover a single event.** Load its .hmbgame file or reopen it from local history. Match code, date, cards and progress before using the printed set.
- 4. Relink moved music.** In CHECK MUSIC select the missing song and Locate selected song.... Choose the same recording. The app checks readable duration and preserves the track ID and card references.

Profile item	What it contains
hazzmusicbingo.db	Library, local games, tracks, cards and progress.
card-design.json / audience-design.json	Current card and audience styles, including individual text styles.
playback-settings.json	The last used clip start offset; separate from game saves and buttons.
game-shortcuts.json	Eight button definitions, themes and resumed-game references.
game-buttons/	Owned save snapshots and background copies. Music remains outside this folder.

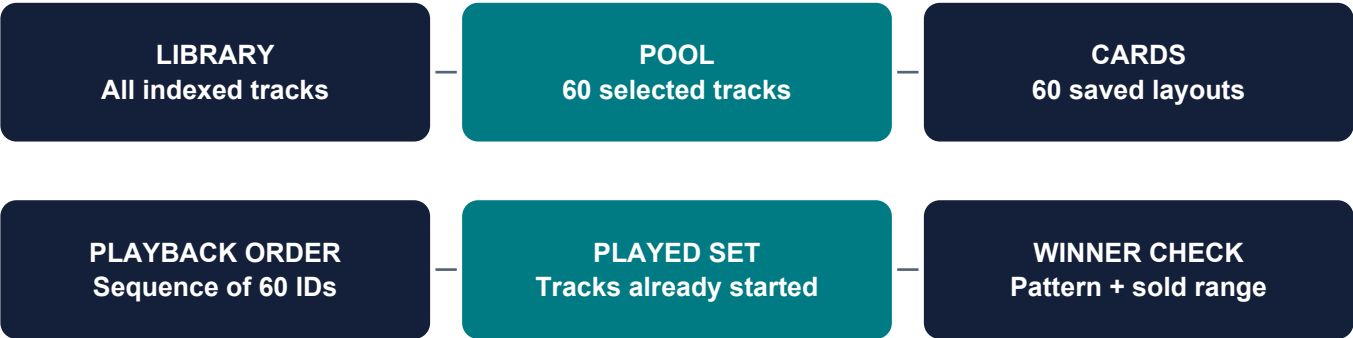
Restoring a whole database

Close the app and preserve the current profile. Restore a backup as hazzmusicbingo.db into a new, empty profile folder; do not mix it with old SQLite WAL/SHM sidecars. Use the advanced profile override documented in the online guide, verify the result, then decide which profile to keep.

Button artwork paths and music paths are machine-specific. When moving computers, relink music and reassign themes/buttons as necessary. For a single event, a portable save plus its music and theme files is usually easier than transplanting an entire profile.

Four different things that look random

Knowing these layers explains what stays the same when you return to a game.



Layer	Plain-English explanation
Pool selection	Choose which 60 track identities belong to this game. The folder filter and availability rules are applied first.
Playback order	Choose the sequence in which the pool is called. This is saved separately from the card layouts.
Card layout generation	Shuffle a list of the 60 tracks, choose 25 distinct offsets, then rotate those offsets for the 60 card numbers.
Winner evaluation	No new randomness: compare saved squares with the played track IDs and the selected pattern.

A title is not the identity

The database gives each indexed track an ID. Cards and progress reference that ID. Two differently named copies of one recording can be separate IDs; changing a title tag does not create a new identity at the same stored path.

The diagrams and equations on the following pages describe the shipped v1.0 code. Numbered tokens stand for tracks in a shuffled list, not literal song titles or printed bingo numbers.

What the app shuffles

A fresh order is different from a fresh card set.

- 1. Filter the library.** Keep songs inside the selected folder scope, then keep files that exist and have a positive scanned duration.
- 2. Choose 60.** Give eligible tracks pseudorandom sort keys, sort them and take the first 60. When exactly 60 qualify, membership cannot change: all of them are selected.
- 3. Store an initial play sequence.** Game creation orders those 60 tracks using another random-key sort. Next Song takes the earliest unplayed item in the saved sequence.
- 4. Generate the cards when requested.** Card generation creates another shuffled permutation of the pool and 25 unique position offsets. The next page shows how those create all 60 layouts.

When does the order change?

Reset to zero deliberately generates a new playback order. Loading a portable saved game also reshuffles order while preserving which songs were already played. Reopening history and returning to an already-loaded game button preserve order. Printing again does not reshuffle saved cards.

Pseudorandom, not a certified lottery mechanism

The implementation uses .NET Random.Shared and random-key sorting. It is designed for a varied hosting experience, not cryptographic unpredictability. Random-key ties can introduce a tiny ordering bias, so the probability examples later use an ideal uniformly shuffled order as a model.

The app does not guarantee that a new game has a different pool or first song from the previous game. Chance can repeat choices. Nor does it scan audio to ensure two selected files are different recordings.

The rotation rule behind every card

One shuffled song list + 25 distinct offsets + 60 rotations.

The actual rule, in words

For each square, start at its chosen offset in the shuffled 60-song list. Add the card’s zero-based index. If you go past 59, wrap to the beginning. Put the song at that position into the square.

Formula: song at square *p* on card *c* = $P[(O[p] + c) \bmod 60]$.
P is the shuffled song list. **O** is the list of 25 different offsets. **c** is 0 for printed card 001, 1 for card 002, up to 59 for card 060. **mod 60** means keep the remainder after dividing by 60.

Square offset	Card 001: +0	Card 002: +1	Card 003: +2
0	P[0]	P[1]	P[2]
7	P[7]	P[8]	P[9]
59	P[59]	P[0]	P[1]

Why this guarantees different songs in one card

The 25 offsets are all different. Adding the same card index and wrapping cannot turn two different offsets into the same result. The shuffled song list also has 60 different track IDs. Therefore each card contains 25 distinct IDs.

Why a square never repeats a song across cards

Hold one square position fixed and increase the card index from 0 to 59. Its offset visits every position in the 60-song list exactly once. That means the top-left square, for example, contains each song exactly once across the complete set.

After construction, the app also checks unordered song sets. If two cards contain exactly the same 25 IDs, even in different positions, it rejects that whole candidate set and builds another before saving.

See the wraparound happen

A six-song teaching model. The real app uses 60 songs and 25 squares.

Imagine the shuffled list is **A, B, C, D, E, F**, at positions 0 to 5. Choose three offsets: **0, 1 and 3**. Add 0 for the first card, 1 for the second, and so on; wrap after position 5.

Card index	Offset 0	Offset 1	Offset 3
0: first card	A	B	D
1	B	C	E
2	C	D	F
3	D	E	A
4	E	F	B
5: sixth card	F	A	C

Read down any column: each of the six songs appears once. Read across any row: three different songs. Across the whole table, each song appears three times - once per square position. The real game does the same thing with 25 positions and 60 rows.

Two illustrative 5 by 5 layouts

00	07	14	21	28
35	42	49	56	03
10	17	24	31	38
45	52	59	06	13
20	27	34	41	48

Card 001: list positions

01	08	15	22	29
36	43	50	57	04
11	18	25	32	39
46	53	00	07	14
21	28	35	42	49

Card 002: each position +1

Here 59 becomes 00 on the second card. These numbers illustrate positions in a shuffled list, not the labels printed for players. This sample offset choice is for teaching only; the app chooses its offsets randomly.

What the guarantees do - and do not - mean

The full set is balanced; the sold subset need not be.

Quantity	Calculation	Result
Total squares in all cards	60 cards × 25 squares	1,500
Appearances of each song	One appearance in each of 25 positions	25
Cards containing a given song	At most once per card; 25 total appearances	25 of 60
Possible lines per card	5 horizontal + 5 vertical	10
Songs needed for a full house	All squares; no free centre	25

A called song marks exactly 25 squares across all 60 cards, on 25 different cards. No two cards mark the same square position on that call, because that position contains the song only once across the set. Two cards can still complete the same indexed row on the call if their last missing squares are in different columns.

This does not guarantee one winner

Cards share songs and their layouts are related by rotation. Different patterns can finish together, and full houses can tie. The program detects qualifying card numbers; your house rules decide how to award prizes.

If you sell only cards 1-42, there is no guarantee that every song appears equally often in those 42 cards. The exact balance is a property of all 60 layouts. Likewise, there is no promise that consecutive card numbers are more or less likely to win in a particular game.

Do not treat the cards as independent random samples. The construction deliberately imposes relationships between cards. The duplicate-set check guarantees different unordered 25-song sets, not zero overlap between them.

From played songs to a winning card

The decision is a direct check, not a prediction.



- 1. Build the played set.** Collect the track IDs recorded as played. Repeating a track does not add another distinct ID.
- 2. Mark each saved square.** For a candidate card, a square is marked if its track ID is in that set.
- 3. Apply the current rule.** Line: all five squares in any row or column. Corners: positions 0, 4, 20 and 24 in row-by-row numbering. Full House: all 25.
- 4. Restrict automatic results.** Only cards from the applied first-to-last range are considered. Manual checking can inspect outside it.
- 5. Track acknowledgement.** Store which rule/card pairs have already been acknowledged. A known winner stays a winner, but does not demand repeat acknowledgement.

Switching rule keeps the evidence

Suppose 32 songs are played when you change from Line to Four Corners. The new rule is tested against all 32 songs immediately. It does not wait for four more songs, clear the grid or start a second round.

The played flags and layouts, not what the audience hears or marks with a pen, determine the result. If sound fails after startup, the track may already count. Use the played list and repeat controls to keep the room aligned with the recorded state.

At most $60 \text{ cards} \times 25 \text{ squares} = 1,500$ square-membership checks are needed for the basic marking pass, plus pattern tests. That small workload makes immediate updates practical on a normal Windows host.

How likely is a completed pattern?

These are model probabilities for one fixed card under an ideal uniformly random playback order.

After **k** different songs have been played from 60, the chance that a particular required set of **m** songs has all appeared is **C(k, m) / C(60, m)**. Here C(n, m), read “n choose m”, counts the ways of selecting m items from n without caring about order. If k is smaller than m, the chance is zero.

Songs played	Four Corners	One specified line	Full House
20	0.994%	0.284%	0.000%
30	5.620%	2.609%	<0.001%
40	18.741%	12.048%	<0.001%
50	47.228%	38.794%	0.243%
55	69.941%	63.696%	5.944%
60	100.000%	100.000%	100.000%

Four Corners has one required set of four songs. **One specified line** means a particular row or column, not any of the ten lines. **Full House** needs that card’s entire 25-song set.

Do not multiply a probability by 10

Lines overlap. Ten times the single-line probability gives the *expected number of completed lines*, not the exact chance of at least one line. The chance that somebody wins among many cards is also not an independent-card calculation: cards share songs.

Expected marks on one card: $25 \times k / 60$. After 30 songs, that is 12.5 marks on average, not a promise that every card has 12 or 13.

Expected full-house call for one card: $25 \times 61 / 26 \approx 58.65$. This is the average position of the last of its 25 songs under the model. The first winner among many cards is a different question. The actual implementation uses pseudorandom-key sorting, so these values describe the ideal model rather than a certified draw distribution.

Timing and practical decisions

There is no fixed “game lasts 30 minutes” promise.

Example	Clip time only
40 calls × 20 seconds	800 seconds = 13 minutes 20 seconds
45 calls × 25 seconds	1,125 seconds = 18 minutes 45 seconds
60 calls × 25 seconds	1,500 seconds = 25 minutes
60 calls × 30 seconds	1,800 seconds = 30 minutes

Add time for marking, chat, repeats, checking winners, prize presentation and any breaks. Short recordings can finish before the selected clip length. Full House timing depends on which cards are in play and the order of their songs.

Before you promise prizes

Decide how many cards each person may use, whether a card can win multiple prizes, what happens on simultaneous claims, how late claims are handled and whether song titles will be shown on screen. Announce the current rule clearly.

Use the display to suit the room

The audience screen normally reveals the current song and can show the played list. That helps players follow along but changes how much recognition is required. Plan your presentation around the app’s actual display; it is not a separate hidden-answer quiz mode.

Rehearse with the real equipment

Use the event speakers, printer and display at their intended resolution and scaling. Run a short practice, switch themes, show the played list and verify a card. The automated tests do not replace that rehearsal.

Troubleshooting during preparation or play

Check the simplest cause first; protect the current game before changing it.

Problem	What to do
Too few songs to generate	Check the source path and subfolder option. Scan first, reconnect the drive and check availability. The selected source needs 60 usable tracks.
Wrong decade in a new game	Confirm the displayed source, especially after restart. Existing saved games retain the pool they were created with.
MP3 titles look wrong	Correct the original tags with your normal tag editor, then rescan before printing. Keep TagLibSharp.dll beside the EXE. Other formats still use filenames.
A song cannot play	Check the file and Windows sound output. Rescan/relink the same recording. Startup failure leaves it unplayed; later failure may not.
No live winner appears	Check that cards were generated, a sold range was Applied, the rule is correct and the required songs are recorded as played.
Unexpected winner number	Check sold range and printed game code. Manual checking can see unsold cards; duplicated paper copies share one number.
Button loads old progress	Buttons resume their local game. Reset deliberately for a replay, or assign a different saved snapshot for a different start.
Background/font missing	Select the artwork again and check the font is installed. Ordinary presets store image paths; button assignments own a copy.
Card/PDF layout is wrong	Match paper/orientation, use larger cards, check printer margins and print a sample. Enable a PDF printer if it is absent.
Settings/history seem absent	Check the Windows account and profile override. Do not replace a database while the app is open.

For repeated failures, note the app version, exact message and steps taken. Keep a copy of the relevant save and backup. Avoid resetting or regenerating a game whose printed cards are already in players' hands.

Event-day checklist

A final check before guests arrive, and a simple running order.

Before doors open

- ☐ The correct game code and song pool are selected.
- ☐ All music files are available; titles were checked before printing.
- ☐ Printed cards match this game; there are no unintended duplicate copies.
- ☐ The first/last sold numbers are entered and Applied.
- ☐ LINE (or your announced first rule) is visibly selected.
- ☐ The speaker output, clip length and Repeat / Stop controls were rehearsed.
- ☐ The audience theme, rule message and played-list pages are readable.
- ☐ The prepared .hmbgame file and database backup are saved.

While hosting

Play → pause for marking → review claims → verify code/card → acknowledge → choose the next rule.

Keep the same played-song progress through your prize sequence. Stop sound yourself if needed; do not assume a winner alert pauses playback.

At the end

Save any checkpoint you want to keep. Close the game rather than deleting its data. Back up the database and the separate button/theme assets. Record which cards and game code were used.

A short player briefing

“Mark a song when you hear it and it appears on your card. We are starting with any horizontal or vertical line. There is no free centre and diagonals do not count. Call bingo and give us your card number. Keep your marks when we move to the next prize.”

Glossary, sources and limits

A concise reference for the terms used throughout this manual.

Term	Meaning
Library	All indexed files, including unavailable entries.
Pool	The 60 track IDs locked to one game.
Permutation	A rearrangement of the same list, with no new items.
Offset / modulo	A starting list position / wraparound after the list's end.
Played set	The distinct track IDs whose playback started successfully.
Snapshot	The state captured when you saved a game or assigned a file.
Session code	The printed game identifier; not an anti-fraud guarantee.
In-play range	The inclusive saved card numbers monitored automatically.

Source of the explanations

Behaviour was checked against the v1.0 source: `GameService`, `CardGenerator`, `WinnerService`, `LiveWinnerService`, `GameFileService`, `GameShortcutService`, `TrackMetadataReader`, `AppDatabase` and the WPF views. The probability examples are derived in this manual under the assumptions on page 27; they are not simulation results or promised event outcomes.

[Release source](#) • [Online user guide](#) • [Downloads](#)

Compatibility and validation

Version-1 game files remain supported when valid. Older saves can default to Line with no sold range. v1.0 passed 410 regression checks and its packaged self-test, including MP3 tags, database, card generation, print rendering and WAV decoding. Real speakers, printer drivers and multi-monitor behaviour still need your rehearsal.

Copyright and included components

Copyright © 2026 Hazz Karaoke. The app licence permits free use, including commercial shows; modified redistribution requires permission under its terms. Third-party licences remain applicable. Read the full [LICENSE.txt](#) and [third-party notices](#) supplied with the package. No music is supplied. This manual describes app operation; it does not grant rights to recordings or artwork.